

Data Security and Privacy

GROUP 10

- Connor Blaha
- Alex Tregub
- Brandom Robare
- Albaraa Alluhaibi
- Meghan McCracken
- Cody Sherman

CONTENTS

- K-Anonymity: A background
- K-Anonymity-Motivations/Real World
- K-Anonymity issues/problems
- Pseudocode
- Implementation
- Demo

BACKGROUND

What is K-Anonymity?



K-ANONYMITY: DEFINITION

- A privacy preserving technique
- Ensures a record is indistinguishable from at least $k-1$ other records, based on their quasi-identifiers.
- Utilizes generalization and suppression for Q.I. and sensitive values
- These aspects help to prevent re-identification of an individual's identity.

K-ANONYMITY: MATHEMATICAL DEFINITION

- Let there be a table of $T(A_1, A_2, \dots, A_n)$, where A is an attribute, or column, and said table has a finite number of tuples represented by rows, each correlating to an object O_i . Each tuple has a value V_i in each of these columns. Each object only has one tuple associated with it. So thus:

	A_1	A_2	A_3
O_1	V_1	V_2	V_3
O_2	V_4	V_5	V_6
O_3	V_7	V_8	V_9

MOTIVATION/REAL WORLD

Real-World Applications Of K-Anonymity



K-ANONYMITY: MOTIVATION & REAL-WORLD CASE

- Prevents identity exposure from quasi-identifiers
- Required by laws like HIPAA and GDPR
- Balances privacy and data usability

To achieve K-anonymity, data holders must be able to identify ‘quasi-identifiers’ or attributes in the table which may be used in conjunction to form a unique identifier for an entry, for example the set of {birth date, zip code, and gender}, and sufficiently generalize or remove (sanitize) attributes as needed such that for there exists K of such identifiers in the dataset. For example, let's take a subset of our dataset, which looks as follows:

```
...
{02-28, 44236, M, ...}
{02-15, 44044, M, ...}
{02-01, 44090, M, ...}
```

...
Suppose all data following our quasi-identifiers are sensitive data which we must protect. If our attacker has access to the following dataset, for example from voter records in the area.

```
...
{Bob, 02-28, 44256, M}
{Evan, 02-15, 44044, M}
{Steve, 02-01, 44090, M}
```

...
In the current form, we can directly associate Bob, Evan, and Steve with their respective attributes in our database. However, if we use ‘generalization’ where we make our data coarser (removing information), our same subset will look like:

```
...
{02-**, 44***, M, ...}
{02-**, 44***, M, ...}
{02-**, 44***, M, ...}
```

...

Pseudo Code Explanation

ALGORITHM DESIGN

- Our goal was to use clustering to partition the data into clusters of at least k size. Clusters can be bigger than k in size but not smaller so we can achieve k anonymity
- We took inspiration from Mondrian and hierarchical clustering.
- Mondrian starts with one large list of points, sorting and partitioning it until each partition cannot get smaller than k .
- In hierarchical clustering, you start with many small clusters, merge them cluster by cluster using a distance metric
- We choose one cluster off of some user defined metric, it could be all the points sorted, pick the smallest, vice versa, or random
- That cluster Greedily consumes the two nearest single point clusters until it has k size.
- Take that cluster off the list, start with a new one point cluster, do the same

PSUEDOCODE

Start with a list of points

Choose a point in the list of clusters to start off as the first glutton cluster

Find the $k-1$ closest neighbors using some distance metric and add those to the glutton cluster, recalculating the centroid of the cluster as they're added. Centroid is calculated as the center of the cluster using a user defined metric.

Once a glutton cluster has k points in it, remove the points from the list of points. Make sure to add that glutton to the list of gluttons

Repeat the process until no points are left in the list.

Check if the last glutton added has less than k points, if so, take points out and distribute them to the next closest gluttons.

Last step is to take all the QI identifier data in each glutton cluster and convert them into a range. Example if age is the quasi-identifier, we take the min age and the max age for the glutton cluster and convert all ages to (min age-max age) thus achieving k anonymity.

APPLICATION

Implementation Details

GENERATING DATASET TO ANONYMIZE

- As our goal is to mimic a real-world application, we have randomly generated some common sensitive and quasi-sensitive attributes for some sample datasets of size 20, 1000, and 200,000
- Our initial dataset of random data contains unique entries representing user data with:
 - 'uid' - for validating results, not used for association
 - 'name' - randomly generated name using 'namemaker' library
 - 'age' - randomly generated integer between 1 – 100
 - 'gender' - character M or F
 - 'zip' - randomly generated 5 digit sequence
 - 'disease' - randomly chosen from list of 11 predefined diseases
- After we generate some number of unique entries, we then create two datasets – one for the client to anonymize, and one for an attacker to use with an association attack.

GENERATING DATASET TO ANONYMIZE CONTINUED.

- Our sample datasets include both up to 10% non-overlap and a 50% non-overlap wrt. the client dataset – where the attacker dataset may not contain all entries in the client dataset, and where the attacker dataset may contain entries not in the client dataset.
 - This is performed by initially generating 1.1 or 1.5 times the target dataset size, and then for the client and attacker datasets, randomly sampling the desired number of entries.
- Then, before we export the datasets, we remove the 'Disease' property from all entries in the attacker's dataset – this will be our attacker's goal: to use the quasi-sensitive attributes to associate a name with a disease.
 - Additionally, the names are not initially removed from the client dataset, however, are kept until after we process them.
- The 'uid' column's use will be described later. In a real-world application, the client would not publish such unique and identifying data along with anonymized data, as this would result in de-anonymization.

EXISTING MONDRIAN ANONYMIZATION ALGORITHM

- For our anonymization algorithms, we define our Quasi-Sensitive attributes which will be anonymized as the 'age', 'gender', and 'zip'. These will be converted to ranges of values – e.g. age = 5-25, zip = 34745- 46899
- For comparison to our clustering-based approach, we also implemented the existing recursive Mondrian algorithm for anonymizing data.
 - The 'Mondrian' function takes a subset of data, a k anonymization value for the stopping condition, and a target anonymization variable.
 - Before proceeding, we check if we have enough subset entries to split the data evenly into 2 datasets. If not, we return this grouping of entries. Otherwise:
 - We sort the subset based on our target variable, then split our current subset into two subsets a low subset and a high subset.

EXISTING MONDRIAN ANONYMIZATION ALGORITHM CONTINUED.

- We will then choose the next variable for the next step. Currently, this is implemented as incrementing the index to our defined list of Q.S. variables.
- We then recurse our Mondrian function on the low and high subsets, adding all final groupings to a list which we will process afterwards.
- After the Mondrian algorithm completes, for all groupings, we will then consider the 'minimum' and 'maximum' for each of our Q.S. variables, which we will then combine into a range of min-max and store this value for all entries in the grouping, thus achieving anonymization through generalization.
- This anonymized data can then be previewed in our web interface alongside being stored locally for our performance evaluation (simulated association attack).

THE 'GLUTTON' ANONYMIZATION ALGORITHM

- We have also developed a greedy clustering-based approach to the anonymization problem. Instead of approaching the data recursively – which requires repeated splitting and sorting based on variables thus building the anonymized data top-down – we instead build the anonymized dataset bottom up, creating a nearest-neighbor clustering of at least k points in each cluster.
 - This approach is based on representing our Q.S. attributes as points in an n -dimensional space where n is the size of the set of Q.S. We can then define a 'distance' between them as the Euclidean distance between two entries.
 - We initially start off by choosing a random seed-point to be our centroid, and then populate the cluster with nearest points, updating the centroid as we go. We repeat this until we have less than K -points remaining, where K is our defined anonymity value – defined as our outlier values.

THE 'GLUTTON' ANONYMIZATION ALGORITHM CONTINUED.

- The handling of these outliers does however affect our algorithm's performance. We have considered two basic approaches: 1) Merging every outlier ($<K$) into the nearest existing cluster OR 2) Discarding all outliers.
 - Our implementation merges outliers into existing clusters, this allows us to keep all client data intact, however, impacts our data's utility negatively as the new ranges for Q.S. variables will be larger than the Mondrian approach or if we were to instead discard the outliers. We believe that this is a worthy tradeoff when considering user privacy, as it will increase how many entries will be similar. Additionally, these outliers can be avoided by choosing a K such that it evenly divides the dataset size, thus allowing the user to increase the utility using the K parameter.
 - Discarding outliers is another approach, increasing utility but decreasing anonymity, as ranges for Q.S. variables will be tighter, thus potentially allowing for easier identification. This is partially mitigated by our age ranges not being fixed, where some may overlap with others, maintaining a larger maximum K .
- Similarly to the Mondrian algorithm, we take the min and max values for each attribute in our clusters and store them instead as ranges, also anonymizing data using generalization.

VERIFYING K-ANONYMITY

- To validate our algorithm's results, we have also implemented an association attack which is performed to attempt to match a potential entries name to a disease, thus de-anonymizing the data using the quasi-sensitive attributes.
 - To do this, we use a brute-force approach, where for each entry in the attacker dataset, we will check through all entries in the client's anonymized dataset to see which ones lead to potential matches. We also check whether any results exist within this data, as potential membership or lack thereof is also a potential privacy exposure.
 - We report the following metrics from the attack : minimum, average, and maximum K value, these are calculated based on how many entries exist in ranges.
The number of users in both datasets – this is checked against the full dataset and client datasets using UID values.
The known number of users not found in client data, where no potential results were found for a user in the attacker's dataset.
And, the maximum and average probability of association – using the full dataset to verify the probability of 'guessing' the correct disease based on the results of the association attack. If no results exist, then using the knowledge that the client dataset contains 11 diseases, and non-membership is a 12th potential property, we add the probability of $1/12$ to the probability of association for that entry.

VERIFYING K-ANONYMITY CONTINUED.

- Each of these metrics reported by the association attack informs us about the performance and success of our algorithms.
 - The minimum k value remains to ensure we have achieved our target K , and our average exists to show what the average identifiability is for any random entry in the attacker's data. The maximum also shows potential utility losses if it is too large, or potential privacy losses if too small.
 - The precise and not found user counts give us the following: as we know the size of the dataset, we can subtract the intersection count and the size giving us the expected number of users not found. For de-anonymized data, this will be equal to the known not found value. For anonymized data, the percentage reduction implies stronger privacy, as membership in the dataset is not necessarily public data, thus if non-members appear to be in the dataset, this will cause issues for the attacker.

VERIFYING K-ANONYMITY CONTINUED.

- Finally, the maximum probability of association can help identify a potential homogeneity attack, as we compare the most likely result to the actual value, if the value is too high above the average, we know that some data may need to be removed or further anonymized.
- The average probability allows us to gain a sense of the anonymization of the data, where while we cannot achieve a perfect score of true random without sacrificing all utility, this can allow the client to fine-tune the K-parameter to achieve their desired tradeoff between privacy and utility for their dataset.
- This approach to testing the anonymity of data can continue to be utilized with other types of anonymization, such as introducing noise to prevent attacks at the cost of utility, and more. This attack should be used in conjunction with both anonymization algorithms to ensure the desired privacy is achieved.

Thank you

PEYTON DAVIS

424-555-0124

PEYTON@PROSEWARE.COM

WWW.PROSEWARE.COM